

Notebook and experiment operation

The two research notebooks are generated from readable Python modules. Edit the modules, run the relevant tests, then rebuild the notebook; do not edit the generated embedded source cell by hand.

Current segmentation protocol: the first case-ID split contained rescaled image copies across roles, so its pilot was cancelled. The completed version-2 pilot and current main study use frozen group-aware partitions (**b92f9c7c0fb4c116**) and fresh weights. The runner rejects a case-ID-only split. Classification version 2 is unaffected.

Notebook 01 — classification ablation

File: notebooks/01_classification_ablation.ipynb.

Private Kaggle notebook: [MRI Classification Ablation v2](#).

It audits the attached yes/no dataset, freezes five nested similarity-group folds, and trains seven variants with seeds 42, 43 and 44: 105 model/fold/seed runs. The maximum schedule is 60 epochs with validation-based early stopping. Training uses mixed precision on a compatible GPU, cached preprocessed images, deterministic batch/augmentation seeds and AdamW. Validation and test use no augmentation. No target accuracy is assumed.

Variant	Interpretation
full	Original multiscale residual classifier with channel/spatial attention
no_attention	Same dilation branches, attention removed
no_multiscale	All three branch dilations set to 1; parameter count preserved
no_attention_no_multiscale	Joint factorial control
no_augmentation	Full model, training augmentation disabled
plain_cnn	Original simple convolutional baseline
narrow_resnet_gn	Original narrow GroupNorm ResNet-style baseline, trained from scratch

Primary reporting averages the seed-specific pooled OOF macro F1 values. It does not average test predictions across seeds and then silently label the resulting ensemble as a single model. Group bootstrap resamples the same image groups for each model and retains every seed inside each draw. Its uncertainty is descriptive because patients are unknown and CV training sets overlap.

Key outputs under `classification_results/` are `protocol_lock.json`, `split_manifest.csv`, `audit/`, `per_run_metrics.csv`, `oof_predictions.csv`, `per_seed_oof_metrics.csv`, `model_summary.csv`, `descriptive_group_bootstrap.csv`, `paired_ablation_deltas.csv`, `histories`, `checkpoints` and `comparison figures`. `completion.json` is required to establish that all 105 runs finished.

CPU preparation for notebook 02

Private staging script: [MRI Segmentation Preparation v2](#).

This is a supporting CPU script job, not a third research notebook. It attaches the MSD mirror, validates all labeled volumes, records source metadata and hashes, freezes case splits, reorders channels, and builds reusable cache files. CPU preparation preserves GPU quota.

The output `prepared/` contains the dataset manifest, source metadata, verified channel permutation, legacy case partitions, version-2 caches and `preparation_complete.json`. A failed integrity gate stops training. Caches with unverified old channel order are rejected. Notebook 02 attaches this private kernel output and reuses its caches, while replacing its failed case-ID partitions with the frozen group metadata embedded from `segmentation/configs/grouped_v2`. It validates the manifest and split hashes before training.

The supporting CPU similarity job uses `scripts/audit_segmentation_similarity.py`; its corrected version 2 produced the group map. `scripts/freeze_segmentation_groups.py` verifies the audit and freezes all four folds,

excludes the cancelled pilot's exposed groups from test eligibility, and selects fixed representative lists. It refuses to silently replace an existing split with a different hash. Both supporting jobs are scripts, so the research package still has two notebooks.

An additional supporting [CPU training-cache job](#) creates lossless eight-slice archives from the canonical caches. The extended runner requires its completed per-case inventory, matching manifest, all expected archives and successful voxel-equality checks. Training reads only the necessary chunks; full-volume inference retains the original caches. Local exact comparisons cover pixel values, labels, padding, slice/group sampling, availability, augmentation and quality. Exactness checks and a completed, bounded Kaggle training benchmark are in `results/optimization/`. That benchmark measured QMMF 4.02× and HeMIS-style 3.51× training-segment speedups; it is not a repeated complete-fit benchmark.

Notebook 02 — segmentation development study

File: `notebooks/02_segmentation_study.ipynb`.

The pilot compares QMMF, HeMIS-style and no-quality variants over eight epochs with 40 microbatches per epoch. It uses fold 0, seed 42, a five-slice context, 192x192 training crops, batch size 4 and gradient accumulation 2. The completed pilot showed low, uncertain scores after short training. Amendment 7 freezes the main study at nine variants × three seeds (42, 43, 44), up to 120 epochs × 100 microbatches, with 12 inner representatives and validation every 10 epochs. Always pass those explicit command-line values: the runner retains older defaults for historical reproducibility. The canonical notebook now contains the main seed-42 execution cell.

The nine-variant matrix adds U-Net 2.5D, no variance, no max, no consistency, shuffled quality and a parameter-matched moment-fusion control. Component flags are applied to the whole configuration before model construction, training and checkpoint loading. All compared variants receive the same case partition and availability curriculum unless that variable is the ablation.

Effective objectives differ across architecture families: QMMF and its six component controls have two auxiliary deep-supervision heads, weighted 0.5 and 0.25; HeMIS-style and U-Net have none. The shared configuration flag does not create missing heads. Cross-family comparisons cover the complete model and training setup, and total training losses across those families cannot be compared as the same objective. The same-family quality and matched-moment controls retain QMMF's auxiliary heads. `scripts/audit_effective_objectives.py` verifies these facts against the frozen submitted source using synthetic inputs and confirms them in the real pilot histories; results are in `results/implementation_audit/`.

Training samples groups uniformly, then cases within groups uniformly. The quality normalizer fits robust statistics across within-group median descriptor matrices, with the exact training case/group IDs saved. The shuffled-quality control assigns donors from a different training group. A checkpoint stores the same normalizer. The teacher receives the full augmented image; the student receives a masked subset. Mixed-precision reductions that can underflow or overflow use float32. Non-finite model outputs fail explicitly.

Pilot checkpoint selection uses four fixed inner-group representatives and three fixed modality subsets every four epochs. The composite rule is 0.6 times full-modality Dice, 0.3 times mean selected-subset Dice and 0.1 times worst selected-subset Dice. This small validation cohort limits ranking stability. The main study uses 12 fixed inner-group representatives, validation every 10 epochs and patience of eight validation checks, with the same composite rule.

The subset reductions happen within each representative case first: take that case's mean or minimum across the three selected subsets, then average across representatives. The minimum is case-specific, so different cases can contribute different worst subsets. This is the checkpoint-selection calculation in the frozen submitted source.

Selected checkpoints are evaluated on all 91 outer cases in development fold 0. Within each case, primary macro Dice averages only reference-nonempty regions. Case macros are then averaged within groups and the 55 groups receive equal weight. Reference-empty regions are recorded separately: both-empty Dice is 1, and nonempty prediction against an empty reference is 0; neither contributes to the primary macro. The case-weighted mean is saved separately. All 15 nonempty modality combinations are evaluated on eight fixed outer-group representatives in the pilot; the frozen extended list has 16 groups. Per-case WT/TC/ET Dice, empty-reference bookkeeping, subset tables and exact Shapley values are saved. These values describe the model's

use of MRI inputs; they do not prove causal or clinical importance. The development jobs do not evaluate the 51-group reserved cohort; a separate frozen evaluation runs after all 27 development fits pass independent verification.

Protected refers to the revised experiment's fitting and selection boundary. The supplied notebook's earlier 16-case validation subset overlaps seven reserved groups. `scripts/audit_original_segmentation_exposure.py` reconstructs that history from the preserved source, outputs and matching ordered split hash. Amendment 10 adds an exploratory sensitivity on 15 groups/18 cases entirely outside the original development partition, while retaining the complete 51-group evaluation and every model/seed. It uses existing evaluation tables and adds no training or inference. The fixed qualitative examples remain unchanged; BRATS_143 was in the earlier validation subset.

For all-15-subset reporting, compute each representative's mean and minimum across its 15 case-level macro Dice scores, then average these values with equal group weights. Thus the reported worst-subset endpoint is an average of case-specific minima. Every reported robustness cohort contains exactly one fixed representative per group. The protected evaluation uses the same reduction on its 51 representatives.

The matched-capacity equal-moment control uses widths [24,48,96,168], 1,227,977 parameters versus QMMF's 1,221,465 (+0.53%). This matches parameter count within 2% and channel alignment; operation count and runtime can still differ.

Each seed has a separate private Kaggle identity (`mri-segmentation-study-s42`, `-s43`, `-s44`) with the same embedded source hash. Two T4 devices can run independent configurations. A GPU is not shared by two training workers. An explicit wall-time budget stops this job's own workers, with per-epoch checkpoints available for continuation. Partial matrices retain pending/failed status and cannot become completed result tables.

Key outputs under `segmentation_results/` are environment and data fingerprints, `splits.json`, development cohorts, `protocol_lock.json`, per-variant configurations, per-epoch histories, normalization provenance, selected/last checkpoints, outer case metrics, modality-subset tables, `model_comparison.csv`, and `session_status.json`.

Live execution logs can be read through the installed API's `kernels_logs_stream` method. A plain `kernels_logs` snapshot may be empty during a running session. Logs are progress evidence; the exported history, per-case tables and completion ledger determine the final result. `scripts/collect_kernel.py` monitors one submitted job, retrieves its outputs and verifies the expected source hash before analysis; it never submits additional experiments.

`scripts/watch_kernel_progress.py` reports development epochs and protected progress by model and seed. Protected subset counters record saved subsets; they do not establish that qualitative outputs or completion receipts have passed verification. The watcher only reads an existing job and reconnects its log stream.

After downloading a completed session, `scripts/analyze_segmentation.py` independently verifies those artifacts and produces group-paired intervals, regional/empty-case tables, all-subset scores, Shapley values and learning curves. Its intervals condition on a single training seed and development fold; they do not estimate variation from retraining. It rejects missing cases, inconsistent split/configuration hashes, group leakage, incorrect group weighting, normalizers fitted outside training, invalid quality donors, and incorrect checkpoint selection. A partial session remains partial even when some models have valid results.

```
python scripts/analyze_segmentation.py \
  kaggle_outputs/segmentation_grouped_pilot_v2/segmentation_results \
  --output results/segmentation_grouped_pilot
```

For completed repeated-seed extended studies, `scripts/combine_segmentation_studies.py` requires every declared seed/fold/variant, common training settings, matching frozen partitions and aligned groups. It reports seed-specific values, mean and sample SD across training seeds, group-bootstrap intervals and paired differences. All fixed seeds remain inside each bootstrap draw; seed count does not increase the number of validation groups. Bonferroni intervals cover the declared QMMF-versus-control family separately for each endpoint and remain descriptive under the documented independence/training limitations.

`scripts/report_segmentation_study.py` checks the selection composite, validation schedule and stopping criterion, then exports per-seed inner-validation curves, a training-diagnostics table, a resource summary, and regional mean/seed-SD tables with separate empty-reference counts. For a verified combined matrix it also produces repeated-seed comparison figures and paired ablation intervals showing both unadjusted and

Bonferroni bounds. Every prespecified model and seed is retained. Reaching an epoch budget, selecting the last checkpoint or observing a small final score change does not establish convergence by itself. Recorded fit/evaluation costs exclude preparation and pre-fit normalizer fitting.

```
python scripts/report_segmentation_study.py \
  results/segmentation_study_s42 \
  results/segmentation_study_s43 \
  results/segmentation_study_s44 \
  --output results/segmentation_main --combined results/segmentation_main
```

Frozen protected evaluation

Amendment 9 adds three fixed qualitative examples to this supporting job, using seed 42 for the same four protected models. `audit/qualitative_protocol.json` freezes the hash-selected case IDs, reference-area slice rule, normalized FLAIR window and RAS orientation. `segmentation/scripts/qualitative_panels.py` repeats full-modality inference for those 12 model/case combinations and saves attributed reference/prediction tiles; it performs no fitting or checkpoint selection. `scripts/analyze_qualitative.py` checks their provenance and whole-volume Dice, then produces a comparison figure and attribution file after quantitative verification. The images are illustrative; the reported quantitative cohorts remain unchanged.

`segmentation/scripts/locked_evaluate.py` is a supporting inference script using the unchanged main-study library. It has no training or normalizer-fitting path. `scripts/build_locked_bundle.py` first combines the complete independently verified development matrix, checks the submitted source, and creates a receipt containing parent artifact hashes. On Kaggle, the evaluator checks the attached parents against that receipt, verifies checkpoint configuration, selected epoch and training-normalizer provenance, and then opens the frozen protected cohort. Each subset is saved separately to preserve progress after interruption.

`scripts/run_remaining_study.py` waits for a verified GPU slot before submitting the already frozen seed-44 bundle. `scripts/run_protected_when_ready.py` waits for all three main seeds and sufficient quota, builds the reviewable evaluation bundle, submits it once, collects outputs and calls `scripts/analyze_locked_segmentation.py`. Runtime ledgers record submission attempts; an uncertain API response must be reconciled before restarting a queue. None of these queues selects models from a favorable ranking.

The independent protected verifier checks every declared model/seed, frozen group representatives, parent hashes, training-only normalization, case/regional/group metrics, repeated full-modality inference and all 15 subsets. It computes separate seed SD and group-bootstrap intervals, with the three prespecified QMMF-control comparisons per endpoint. Artificial test fixtures exercise missing seeds, changed cohorts/checkpoints and inconsistent metrics without loading any protected MRI. A successfully built script is not evidence of a completed protected evaluation; the results report and verified completion ledger provide that status.

After that verification, produce descriptive regional, all-subset, Shapley and resource tables:

```
python scripts/report_reserved_results.py \
  results/segmentation_protected \
  --output results/segmentation_protected/reporting
```

The script rechecks regional group reductions and empty-reference counts, preserves all four models and three seeds, and records input/output hashes. Subset and Shapley tables show means and sample seed SD; they add no hypothesis tests. Shapley uses a declared zero empty-set value and describes model utility under the simulated masking protocol, without establishing causal scanner utility or patient-level effects. Worker timers can overlap and do not measure isolated inference latency.

After complete quantitative and nonsynthetic qualitative verification, reproduce the historical audit and run the supplementary sensitivity report:

```
python scripts/audit_original_segmentation_exposure.py
python scripts/analyze_historical_sensitivity.py \
  results/segmentation_protected \
  --output results/segmentation_historical_sensitivity
```

The sensitivity script verifies the frozen 15-group membership, validates the full source cohort before filtering, retains all four models and three seeds, and recomputes the saved full-cohort summaries and paired intervals. It exports separate tables and a comparison figure with input/output hashes. The full 51-group results remain the planned report; the additional subset is exploratory and too small to establish patient-level or external validation.

Rebuild, verify, submit and retrieve

Run from the repository root:

```
PYTHONPATH=segmentation/src python -m pytest tests segmentation/tests -q
python scripts/build_notebooks.py classification
python scripts/build_notebooks.py preparation
python scripts/build_notebooks.py segmentation \
  --scope study --seed 42 --fold 0 \
  --epochs 120 --steps 100 --val-every 10 \
  --budget-seconds 29000 --slug mri-segmentation-study-s42
python scripts/build_notebooks.py training-cache
```

The helper uses `KAGGLE_API_TOKEN` from the environment, or an external local token file specified by `MRI_KAGGLE_TOKEN_FILE`. The token is never a notebook parameter and never embedded in a source snapshot.

```
python scripts/kaggle_api.py quota
python scripts/kaggle_api.py push kaggle/classification --timeout 3600
python scripts/kaggle_api.py push kaggle/preparation --accelerator None --timeout 10800
python scripts/kaggle_api.py status dasshovon/mri-segmentation-preparation-v2
```

Wait for preparation to report `COMPLETE` and verify its manifest against the frozen group metadata before submitting the dependent cache job. A `status` command reads the current state; it does not wait for completion. The classification job is independent of these segmentation prerequisites.

```
python scripts/kaggle_api.py push kaggle/training_cache --accelerator None --timeout 3600
python scripts/kaggle_api.py status dasshovon/mri-segmentation-training-cache
```

Submit the main study only after the training-cache job reports `COMPLETE` and its inventory passes the documented equality and completeness checks. Its metadata must attach both completed preparation and training-cache outputs.

```
python scripts/kaggle_api.py push kaggle/segmentation --timeout 30000
python scripts/kaggle_api.py status dasshovon/mri-classification-ablation-v2
python scripts/kaggle_api.py output dasshovon/mri-classification-ablation-v2 \
  --out kaggle_outputs/classification
```

Change `--user` when rebuilding for another Kaggle account. The official accelerator name is case-sensitive in this workflow: use `NvidiaTeslaT4`. The first submission used an incorrect identifier and fell back to `P100`; its installed PyTorch CUDA wheel did not include that architecture. That failed attempt is retained in the run record. [Official Kaggle kernel commands](#), [PyTorch architecture-support notice](#).

Scope and dependencies

The original classification result table is preserved as historical evidence in the audit. The new table must come from retrieved Kaggle artifacts. Synthetic fixture scores are test output only. The frozen protected protocol requires all 27 development fits to pass independent verification before evaluation. It uses QMMF, HeMIS-style, no quality and U-Net 2.5D for all three seeds, with their existing inner-selected checkpoints. Full evaluation covers 66 cases/51 groups; all 15 subsets use one frozen representative per protected group. No protected-case retraining, scaler fitting or threshold tuning is allowed.

Kaggle installs and local verification can have different package versions. The execution environment fingerprint belongs with every reported table. The source snapshot hash and split/data hashes identify the experiment; source-code tests alone cannot reproduce a learned checkpoint.

Preserved executed-source dictionaries are under `audit/source_snapshots/`; their hashes distinguish submitted code from later improvements in the readable source tree. The current notebook file records the submitted main seed-42 study. Historical pilot and benchmark snapshots remain separate. Rebuilding from later source changes creates a new hash; use preserved snapshots when reproducing a reported earlier version.